

Министерство образования Республики Беларусь

Учреждение образования
БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ

Факультет информационных технологий и управления
Кафедра интеллектуальных информационных технологий

Отчет
к лабораторной работе №2.2
по дисциплине "Проектирование защищенных интеллектуальных
информационных систем"
на тему:

**ОЦЕНКА ЗАЩИЩЁННОСТИ СТОРОННИХ
ПРОГРАММНЫХ СИСТЕМ**

Студент гр. 121702
Руководитель

Д. С. Голушко
В. В. Захаров

Минск 2024 г.

СОДЕРЖАНИЕ

1	Ход работы	6
1.1	Скриншоты выполнения задания	6
1.2	Методика оценки защищенности систем	12
1.2.1	Анализ уязвимостей на уровне кода	12
1.2.2	Аудит прав доступа	12
1.2.3	Тестирование на проникновение	12
1.2.4	Логирование и мониторинг	12
1.2.5	Оценка архитектуры безопасности	12
1.3	Анализ разработанного приложения	13
1.3.1	Политика безопасности операционной системы	13
1.3.2	Политика безопасности компонентов приложения и их взаимодействие между собой	13
1.3.3	Сетевая политика безопасности	14
1.3.4	Безопасность конфиденциальности данных	14
1.3.5	Вывод	14
1.4	Комплекс мер по усовершенствованию приложения	14
1.4.1	Шифрование конфиденциальных данных	14
1.4.2	Улучшение аутентификации и авторизации	15
1.4.3	Регулярные обновления безопасности	15
1.4.4	Логирование и мониторинг событий	15
1.5	Рекомендации по безопасному использованию приложения	15
1.5.1	Использование сильных паролей	15
1.5.2	Регулярные резервные копии	15
1.5.3	Контроль прав доступа на уровне ОС	16
1.5.4	Обеспечение физической безопасности	16
1.5.5	Ограничение доступа к важным ресурсам	16
1.5.6	Регулярное обновление системы	16
1.5.7	Обучение пользователей	16
1.5.8	Вывод	16
	Вывод	17
	Список использованных источников	17

1 ХОД РАБОТЫ

Постановка задачи: Разработать методику оценки защищённости систем.

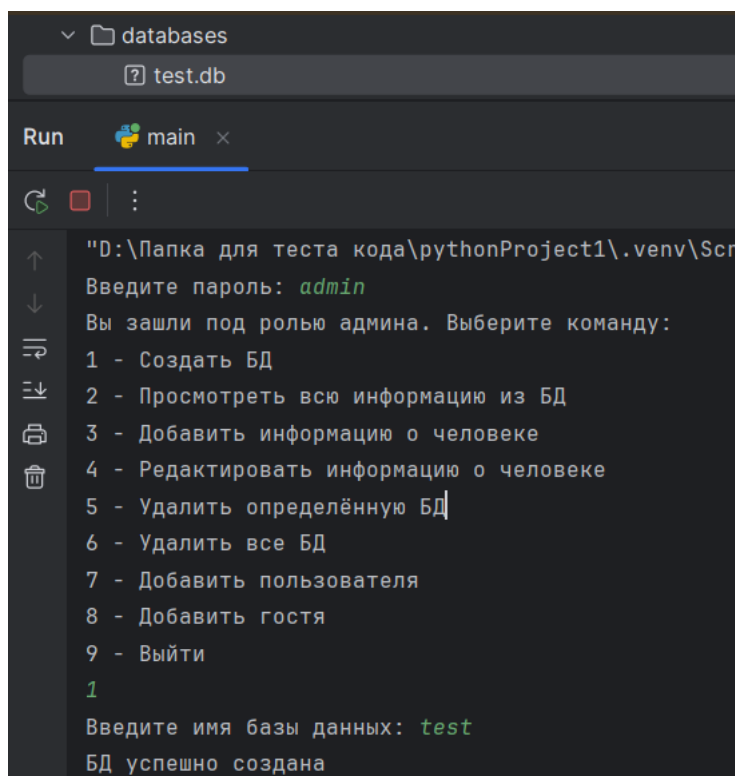
Также необходимо выполнить анализ разработанного приложения с точки зрения:

- политики безопасности операционной системы,
- политики безопасности компонентов приложения и их взаимодействия между собой,
- сетевой политики безопасности,
- безопасности конфиденциальности данных.

Также необходимо предложить комплекс мер по усовершенствованию приложения и предложить рекомендации по безопасному использованию приложения.

1.1 Скриншоты выполнения задания

1. Вход под записью администратора, создание новой базы данных;



```

D:\Папка для теста кода\pythonProject1\.venv\Scripts>python main.py
Введите пароль: admin
Вы зашли под ролью админа. Выберите команду:
1 - Создать БД
2 - Просмотреть всю информацию из БД
3 - Добавить информацию о человеке
4 - Редактировать информацию о человеке
5 - Удалить определённую БД
6 - Удалить все БД
7 - Добавить пользователя
8 - Добавить гостя
9 - Выйти
1
Введите имя базы данных: test
БД успешно создана

```

Рисунок 1.1 – Изображение входа и создания базы данных

2. Проверка базы после создания;

```
Введите пароль: admin
Вы зашли под ролью админа. Выберите команду:
1 - Создать БД
2 - Просмотреть всю информацию из БД
3 - Добавить информацию о человеке
4 - Редактировать информацию о человеке
5 - Удалить определённую БД
6 - Удалить все БД
7 - Добавить пользователя
8 - Добавить гостя
9 - Выйти
2
Введите имя базы данных: test
База данных пуста.
```

Рисунок 1.2 – Изображение проверки базы после создания

3. Добавление нового гостя;

```
Введите пароль: admin
Вы зашли под ролью админа. Выберите команду:
1 - Создать БД
2 - Просмотреть всю информацию из БД
3 - Добавить информацию о человеке
4 - Редактировать информацию о человеке
5 - Удалить определённую БД
6 - Удалить все БД
7 - Добавить пользователя
8 - Добавить гостя
9 - Выйти
8
Введите имя нового пользователя: new
Гость new успешно добавлен.
```

Рисунок 1.3 – Изображение добавления нового гостя

4. Создание записи в базе данных;

```
Введите пароль: admin
Вы зашли под ролью админа. Выберите команду:
1 - Создать БД
2 - Просмотреть всю информацию из БД
3 - Добавить информацию о человеке
4 - Редактировать информацию о человеке
5 - Удалить определённую БД
6 - Удалить все БД
7 - Добавить пользователя
8 - Добавить гостя
9 - Выйти
3
Введите имя базы данных: test
Введите имя человека: САШКА ПИПКО
Введите возраст человека: 20
Введите зарплату человека: -1000
Информация о человеке добавлена успешно
```

Рисунок 1.4 – Изображение создания записи в базе данных

5. Просмотр базы данных;

```
Введите пароль: admin
Вы зашли под ролью админа. Выберите команду:
1 - Создать БД
2 - Просмотреть всю информацию из БД
3 - Добавить информацию о человеке
4 - Редактировать информацию о человеке
5 - Удалить определённую БД
6 - Удалить все БД
7 - Добавить пользователя
8 - Добавить гостя
9 - Выйти
2
Введите имя базы данных: test
Вся информация из БД:
(1, 'САШКА ПИПКО', 20, -1000)
```

Рисунок 1.5 – Изображение просмотра базы данных

6. Создание нового пользователя;

```
Вы зашли под ролью админа. Выберите команду:  
1 - Создать БД  
2 - Просмотреть всю информацию из БД  
3 - Добавить информацию о человеке  
4 - Редактировать информацию о человеке  
5 - Удалить определённую БД  
6 - Удалить все БД  
7 - Добавить пользователя  
8 - Добавить гостя  
9 - Выйти  
7  
Введите имя пользователя: sashs  
Введите пароль пользователя: 1234  
Пользователь sashs успешно добавлен.
```

Рисунок 1.6 – Изображение создания нового пользователя

7. Добавление записи в базу данных пользователем;

```
Введите пароль: 1234  
Вы зашли как обычный пользователь. Выберите команду:  
1 - Просмотреть всю информацию из БД  
2 - Добавить информацию о человеке  
3 - Редактировать информацию о человеке  
4 - Выйти  
2  
Введите имя базы данных: test  
Введите имя человека: NIKITA  
Введите возраст человека: 20  
Введите зарплату человека: 1000
```

Рисунок 1.7 – Изображение добавление записи в базу данных пользователем

8. Просмотр базы данных после действий пользователя;

```
Введите пароль: 1234
Вы зашли как обычный пользователь. Выберите команду:
1 - Просмотреть всю информацию из БД
2 - Добавить информацию о человеке
3 - Редактировать информацию о человеке
4 - Выйти
1
Введите имя базы данных: test
Вся информация из БД:
(1, 'САШКА ПИПКО', 20, -1000)
(2, 'НИКИТА', 20, 1000)
```

Рисунок 1.8 – Изображение просмотра базы данных после действий пользователя

9. Просмотр информации от записи гостя;

```
Введите пароль или логин гостя: guest
Вы зашли как гость. Выберите команду:
1 - Просмотреть информацию из БД
2 - Выйти
1
Введите имя базы данных: test
Вся информация из БД:
('САШКА ПИПКО',)
('НИКИТА',)
```

Рисунок 1.9 – Изображение просмотра информации от записи гостя

10. Обновление информации от записи администратора;

```
Введите пароль или логин гостя: admin
Вы зашли под ролью админа. Выберите команду:
1 - Создать БД
2 - Просмотреть всю информацию из БД
3 - Добавить информацию о человеке
4 - Редактировать информацию о человеке
5 - Удалить определённую БД
6 - Удалить все БД
7 - Добавить пользователя
8 - Добавить гостя
9 - Выйти
4
Введите имя базы данных: test
Введите ID человека для редактирования: 1
Текущая информация: ID=1, Имя=САШКА ПИПКО, Возраст=20, Зарплата=-1000
Введите новое имя (оставьте пустым, чтобы не изменять): ALEX
Введите новый возраст (оставьте пустым, чтобы не изменять): 25
Введите новую зарплату (оставьте пустым, чтобы не изменять): 1234
Информация о человеке с ID=1 успешно обновлена.
```

Рисунок 1.10 – Изображение обновления информации

11. Просмотр базы после изменений;

```
Введите пароль или логин гостя: admin
Вы зашли под ролью админа. Выберите команду:
1 - Создать БД
2 - Просмотреть всю информацию из БД
3 - Добавить информацию о человеке
4 - Редактировать информацию о человеке
5 - Удалить определённую БД
6 - Удалить все БД
7 - Добавить пользователя
8 - Добавить гостя
9 - Выйти
2
Введите имя базы данных: test
Вся информация из БД:
(1, 'ALEX', 25, 1234)
(2, 'NIKITA', 20, 1000)
```

Рисунок 1.11 – Изображение просмотра базы после изменений

1.2 Методика оценки защищенности систем

С целью разработки методики оценки защищенности систем необходимо выделить аспекты, которые необходимо учесть при разработке данной методике, и на их основе создавать методику.

1.2.1 Анализ уязвимостей на уровне кода

Необходимо проверить код приложения на наличие уязвимостей, таких как SQL-инъекции, ошибки в управлении доступом, некорректное управление сессиями и т.д. Использование инструментов статического анализа кода (SAST) может помочь автоматизировать этот процесс.

1.2.2 Аудит прав доступа

Надо провести аудит файлов и папок, которые используются приложением. Проверить, какие пользователи и процессы имеют доступ к данным и каким образом этот доступ ограничен.

1.2.3 Тестирование на проникновение

В данном случае необходимо провести имитационные атаки для проверки того, как приложение ведет себя в случае попытки взлома. Это включает в себя проверку на brute force атак на пароли, попытки получить доступ к закрытым данным и другие возможные сценарии эксплуатации уязвимостей.

1.2.4 Логирование и мониторинг

Требуется оценить возможности приложения для ведения логов о попытках несанкционированного доступа и действий внутри системы. Это важно для отслеживания подозрительных действий и быстрого реагирования на инциденты.

1.2.5 Оценка архитектуры безопасности

Также требуется оценить, насколько хорошо приложение изолирует разные компоненты и уровни доступа пользователей, а также насколько защищены данные, обрабатываемые в системе. Определить, где могут возникнуть проблемы с конфиденциальностью или целостностью данных.

1.3 Анализ разработанного приложения

Выполним анализ приложения в соответствии с требованиями лабораторной работы

1.3.1 Политика безопасности операционной системы

Работа с файлами: Приложение использует файловую систему для работы с базами данных и JSON-файлами. Файлы базы данных и конфиденциальные данные пользователей хранятся в локальных файлах. Важно, чтобы на уровне операционной системы были настроены правильные права доступа к этим файлам (чтение/запись только для администратора), чтобы предотвратить несанкционированный доступ.

При условии, что права доступа к файлам правильно настроены, и никто, кроме администратора, не имеет доступа к этим ресурсам, приложение будет надежно защищено от угроз, связанных с несанкционированным доступом.

Работа с процессами и доступ к системным ресурсам: Приложение не использует системные ресурсы интенсивно, а также не запускает внешние процессы, что минимизирует риски эксплуатации на уровне ОС. Также оно не требует повышения привилегий, что снижает вероятность нарушения безопасности на уровне прав доступа.

Приложение работает в рамках стандартных пользовательских привилегий, что ограничивает возможность эксплуатации уязвимостей на уровне операционной системы.

1.3.2 Политика безопасности компонентов приложения и их взаимодействие между собой

Аутентификация пользователей: Приложение поддерживает несколько уровней пользователей (администратор, обычный пользователь, гость). Аутентификация осуществляется через ввод пароля. Данные пользователей хранятся в JSON-файлах, что упрощает управление ими. Административные функции защищены паролем, и обычные пользователи не могут получить доступ к административным функциям, что укрепляет безопасность.

Правильная разграниченность прав доступа для разных уровней пользователей, а также наличие аутентификации на каждом уровне обеспечивает безопасное взаимодействие компонентов приложения.

Изоляция баз данных: Приложение изолирует базы данных для каждого запроса, используя отдельные файлы. Это снижает риск утечки данных между различными базами. При этом манипуляции с базами данных доступны только авторизованным пользователям.

Изоляция данных и контроль доступа к базе данных усиливают безопасность и гарантируют, что данные остаются конфиденциальными и защищенными от посторонних.

1.3.3 Сетевая политика безопасности

Отсутствие сетевых подключений: Приложение работает в локальной среде и не требует подключения к сети для выполнения основных функций. Это устраняет многие риски, связанные с сетевой безопасностью, такие как атаки через сеть, утечки данных через интернет и т. д.

Отсутствие сетевых взаимодействий исключает сетевые атаки и снижает уязвимость к внешним угрозам.

1.3.4 Безопасность конфиденциальности данных

Хранение данных: Приложение хранит данные пользователей и гостей в JSON-файлах. Для администраторов и пользователей информация защищена через механизм аутентификации паролями. Уровень конфиденциальности данных напрямую зависит от безопасности файловой системы.

Минимизация утечки данных: Приложение предоставляет гостям ограниченный доступ только к именам людей, исключая возможность просмотра конфиденциальной информации о них (например, зарплат или возраста).

Хранение данных в изолированных JSON-файлах и ограничение прав доступа для различных ролей обеспечивает высокий уровень конфиденциальности и защиты данных пользователей.

1.3.5 Вывод

При правильной настройке операционной системы (особенно прав доступа к файлам) и организации контролируемого доступа на уровне приложения, текущая реализация обладает хорошей базовой безопасностью. Благодаря отсутствию сетевых подключений и четкому разграничению прав пользователей приложение может считаться безопасным с точки зрения управления данными и конфиденциальности.

1.4 Комплекс мер по усовершенствованию приложения

1.4.1 Шифрование конфиденциальных данных

Хранение паролей пользователей в открытом виде — это большой риск. Необходимо использовать алгоритмы хэширования паролей (например, bcrypt или Argon2), чтобы защитить пароли от утечек. Шифрование данных, хранимых в JSON-файлах и базах данных (особенно конфиденциальных

данных, таких как зарплата или другие личные сведения), повысит уровень безопасности.

1.4.2 Улучшение аутентификации и авторизации

Реализовать двухфакторную аутентификацию (2FA) для повышения безопасности доступа администраторов. Внедрить тайм-ауты для сессий пользователей, чтобы исключить возможность злоупотребления неавторизованными пользователями при отсутствии активности. Ограничить количество попыток ввода пароля, чтобы предотвратить brute force атаки. Использование безопасных SQL-запросов:

Сейчас в коде используются параметризованные запросы, что хорошо, но нужно убедиться, что SQL-инъекции полностью исключены и данные пользователя корректно проверяются перед записью в базу данных.

1.4.3 Регулярные обновления безопасности

Приложение должно периодически проверяться на наличие уязвимостей с использованием инструментов динамического анализа (DAST). Также должны выпускаться регулярные обновления безопасности для устранения новых уязвимостей.

1.4.4 Логирование и мониторинг событий

Добавить систему логирования для отслеживания всех изменений данных (создание, редактирование, удаление), а также попыток входа в систему. Внедрить мониторинг действий администраторов, пользователей и гостей для быстрого реагирования на подозрительные активности.

1.5 Рекомендации по безопасному использованию приложения

1.5.1 Использование сильных паролей

Необходимо требовать от всех пользователей создания надежных паролей (минимум 8 символов, включающих заглавные буквы, цифры и специальные символы) для снижения риска компрометации учетных записей. Рекомендовать пользователям периодически менять свои пароли.

1.5.2 Регулярные резервные копии

Регулярное создание резервных копий баз данных и JSON-файлов, в которых хранятся данные пользователей, поможет быстро восстановить систему в случае атаки или потери данных.

1.5.3 Контроль прав доступа на уровне ОС

Настроить права доступа на уровне файловой системы для ограничения доступа к базам данных и JSON-файлам только администратору. Это предотвратит несанкционированное чтение или изменение данных.

1.5.4 Обеспечение физической безопасности

Поскольку приложение работает локально, важно контролировать физический доступ к устройствам, на которых оно установлено, чтобы избежать несанкционированного вмешательства.

1.5.5 Ограничение доступа к важным ресурсам

Необходимо использовать сетевые брандмауэры или другие средства ограничения доступа, чтобы к критически важным ресурсам приложения (например, к базе данных) имели доступ только авторизованные пользователи из доверенных сетей.

1.5.6 Регулярное обновление системы

Поддерживать приложение и его компоненты (SQLite, Python и другие библиотеки) в актуальном состоянии, чтобы минимизировать риски эксплуатации известных уязвимостей.

1.5.7 Обучение пользователей

Обучать пользователей правильным практикам безопасности, включая использование надежных паролей, избегание фишинговых атак и недопущение передачи конфиденциальных данных третьим лицам.

1.5.8 Вывод

Для повышения уровня безопасности данного приложения важно учесть меры по шифрованию данных, улучшению аутентификации, а также управлению правами доступа. Соблюдение рекомендаций по безопасному использованию поможет минимизировать риски и повысить защищенность системы.

ВЫВОД

В ходе выполнения лабораторной работы были выполнены все поставленные задания: создана программа, позволяющая работать с пользователями и выполнять различные операции с конфиденциальными и неконфиденциальными данными, разработана методика оценки безопасности. Также выполнен анализ приложения с точек зрения безопасности операционной системы, работы с данными, сетевой политики и работы компонентов между собой, предложен комплекс мер по усовершенствованию приложения и предложены рекомендации по безопасному использованию приложения.